Technical Document – Lab 3

1 – Player Stats and Damage Interface

The PlayerStats.cs script was created inside Assets/Scripts/Player and changed to a ScriptableObject. The CreateAssetMenu attribute was added to allow Player Stats assets to be created through the Unity menu.

The Config and Health headers were added, along with variables for the player's level, current health and maximum health.

A folder called SO was created inside Assets, and a Player Stats ScriptableObject was created and configured in the Inspector with appropriate values.

A folder called Extra was created inside Assets/Scripts, and the IDamageable.cs interface was added. The interface defines the TakeDamage(float amount) method, which can be implemented by classes that need to handle damage.

The PlayerHealth class was updated to implement IDamageable. This made the TakeDamage() method available through the interface, allowing the class to define how the player receives damage.

2 – Player Health System

A private PlayerStats variable was added to PlayerHealth.cs and exposed in the Inspector using SerializeField. This allows the script to access and modify the player's health values stored in the ScriptableObject.

The TakeDamage() method was implemented to subtract the specified damage amount from the player's current health. An if statement was added to check whether the health reached zero or below, calling PlayerDead() if the condition was met.

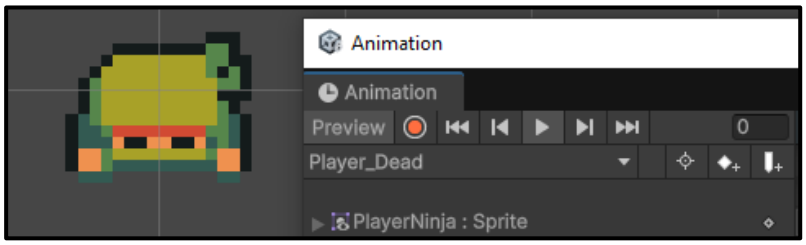
The PlayerDead() method was created to display the message "Jogador Derrotado" in the Unity Console using Debug.Log().

A temporary test was added to Update() using Input.GetKeyDown(KeyCode.P), allowing damage to be applied by pressing P during Play mode. The PlayerStats asset was assigned to the PlayerHealth component, and the damage system was tested.



3 – Player Death Animation

A new animation called Player_Dead was created using the appropriate sprite in a single keyframe at frame zero.



A trigger parameter called Dead was added to the Animator. A transition from Any State to the death animation was created, with the Dead trigger as its condition. Has Exit Time and Fixed Duration were disabled, and Transition Duration was set to zero.

The PlayerAnimations.cs script was created inside Assets/Scripts/Player and added to the player character.

The moveX, moveY, moving and dead variables were created using Animator.StringToHash() to reference the Animator parameters. The Animator component was obtained using GetComponent<Animator>().

Three methods were added to control the animations. SetDeadAnimation() activates the death animation using SetTrigger(), SetMoveBoolTransition() updates the Moving boolean using SetBool(), and SetMoveAnimation() updates the MoveX and MoveY parameters using SetFloat() and the movement direction provided through a Vector2 argument.

4 – Movement and Animation Integration

The PlayerMovement.cs script was updated with a PlayerAnimations variable, initialized using GetComponent<PlayerAnimations>().

The animation logic inside ReadMovement() was replaced with calls to SetMoveBoolTransition() and SetMoveAnimation(). This separates the movement logic from the animation controls and allows PlayerAnimations to manage the character's animation parameters.

A private PlayerAnimations variable was also added to PlayerHealth.cs and initialized in Awake() using GetComponent<PlayerAnimations>().

The PlayerDead() method was updated to call SetDeadAnimation() through the PlayerAnimations component. The TakeDamage() method checks whether the player's health has reached zero or below and calls PlayerDead() when necessary.

5 – Player Stats Access and Movement Restriction

A new Player.cs script was created inside Assets/Scripts/Player. The class contains a private PlayerStats variable exposed in the Inspector using SerializeField.

A public Stats property was added using an expression-bodied property. This allows other classes to access the PlayerStats reference without directly accessing the private variable.

The PlayerMovement.cs script was updated with a private Player variable, initialized in Awake() using GetComponent<Player>().

In the Move() method, a condition was added to check whether the player's current health is less than or equal to zero. If the condition is met, the method returns before calculating or applying movement.

Finally, the Player component was added to the player character, and the Player Stats ScriptableObject was assigned in the Inspector.

The project was tested to verify that the player could receive damage, play the death animation when health reached zero and stop moving when no health remained.